The Journal of Open Source Software

CD-Dynamax: A JAX-based Python package for continuous-discrete probabilistic state space modeling and inference

Matthew Levine [1,2¶], **Daniel Waxman** [2], **and Iñigo Urteaga** [3,4¶]

1 Broad Institute of MIT and Harvard, USA **2** Basis Research Institute, USA **3** BCAM – Basque Center for Applied Mathematics, Spain **4** IKERBASQUE — Basque Foundation for Science, Spain ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

cd-dynamax is a JAX-based Python package for modeling, inference, and learning in ***continuous (time dynamics) - discrete (time observation)* state space models** (CD-SSMs). It provides a low-level, modular library of algorithms for filtering, smoothing, forecasting, and parameter learning in systems where latent states evolve continuously in time according to stochastic differential equations (SDEs) and noisy observations are collected at discrete, possibly irregular, time instants.

cd-dynamax is designed to facilitate the adoption, analysis, and experimentation of CD-SSMs by researchers and practitioners in various scientific domains. It provides (i) flexible CD-SSM model definitions supporting both linear and nonlinear dynamics and observation models, (ii) state-of-the-art filtering, smoothing, and forecasting algorithm implementations, and (iii) tools for system identification and model parameter estimation, all within an efficient, autodifferentiable JAX framework that enables seamless GPU/TPU acceleration and integration with modern inference methods.

Mathematical background

Dynamical systems, often modeled as stochastic differential equations (SDEs), are widely used mathematical tools to describe complex phenomena in various scientific fields, including engineering, economics, neuroscience, ecology, and climate science. In most real-world scenarios, observations of these systems are collected, subject to noise, at discrete and irregular time intervals, requiring nuanced modeling approaches. Continuous-discrete state space models (CD-SSMs) provide a powerful probabilistic framework for modeling such systems (Särkkä & Solin, 2019).

A CD-SSM is described by:

- A (possibly unknown) stochastic dynamical system, i.e.,

$$dx(t) = f(x(t), u(t), t)dt + L(x(t), u(t), t)dw(t) \ ,$$

  where:
  - $x(t) \in \mathbb{R}^{d_x}$ and $x(0) \sim P(x_0)$,
  - $u(t) \in \mathbb{R}^{d_u}$ is an external input (i.e., control) signal,
  - $f : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_x}$ is the drift function,
  - $L : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_x \times d_w}$ is the diffusion coefficient, and
  - dw is the derivative of a d_w-dimensional Brownian motion with covariance Q.

- Data is observed at arbitrary times $\{t_k\}_{k=1}^K$ via a measurement process

$$y(t) = h(x(t), u(t), t) + \eta(t) \ ,$$

  where:
  - $h : \mathbb{R}^{d_x} \times \mathbb{R}^{d_u} \times \mathbb{R} \to \mathbb{R}^{d_y}$ transforms the d_x-dimensional latent state to a d_y-dimensional observation, and
  - $\eta(t)$ is an independent and identically distributed noise process that corrupts the observations.
- The collection of CD-SSM parameters is denoted with θ, which may include parameters of the initial state distribution $P(x_0)$, parameters governing the latent dynamics (e.g., parameters of f, L and Q) and/or parameters of the observation model (e.g., parameters of h and, in the Gaussian observation noise case, its covariance matrix R).

This mathematical framework describes ***continuous (dynamics)*** - ***discrete (observation)*** **state space models**. Under this formulation, CD-SSMs enable accurate modeling of dynamical systems where noisy data are collected at irregular intervals and the underlying processes evolve continuously over time.

For a given set of observations $Y_K = [y(t_1), \ \dots, \ y(t_K)]$ and inputs $U_K = [u(t_1), \dots, u(t_K)]$ of a CD-SSM, where inputs are assumed piecewise constant (zero-order hold) between observation instants, i.e., $u(t) = u(t_k)$ for $t \in [t_k, t_{k+1})$, the main inference and learning objectives are:

- **Filtering**: estimating the distribution of $x(t_K) \mid Y_K, U_K, \ \theta$.
- **Smoothing**: estimating the distribution of $\{x(t)\}_{t \leq t_K} \mid Y_K, U_K, \ \theta$.
- **Forecasting**: estimating the distribution of $\{x(t)\}_{t > t_K} \mid Y_K, U_K, \ \theta$.
- **Parameter learning**: estimating $\theta \mid Y_K, U_K$, either point-wise or in distribution.

Each of the above tasks, when dealing with CD-SSMs with nonlinear dynamics and/or observation models, requires the use of approximate inference algorithms, which in turn require careful implementations to ensure numerical stability and efficiency. More information about state space modeling can be found in the textbooks by Murphy (2023), Särkkä & Solin (2019), and Särkkä & Svensson (2023).

Statement of need

`cd-dynamax` is a JAX-based (Bradbury et al., 2018) open-source Python package that provides a **low-level library** for continuous-discrete state space modeling, inference, and learning.

When modeling complex dynamical systems, scientists face the challenge of accurately capturing the underlying continuous-time dynamics while accounting for the discrete and noisy nature of real-world observations. CD-SSMs provide a powerful framework for addressing this challenge, yet the implementation of CD-SSMs and their associated inference algorithms can be complex and computationally demanding, especially for nonlinear models and continuous-time, stochastic dynamics.

To the best of our knowledge, no existing JAX-based library provides a comprehensive framework for CD-SSM modeling and inference. Existing JAX-native SSM libraries, such as `dynamax` (S. W. Linderman et al., 2025) and `cuthbert` (Corenflos & Duffield, 2026), focus on discrete-time state space models and do not address the continuous-time dynamics formulation. `cd-dynamax` fills this gap by providing efficient implementations of state-of-the-art filtering, smoothing, forecasting, and parameter learning algorithms specifically designed for continuous-discrete dynamical systems, within an autodifferentiable framework that enables gradient-based system identification and modern Bayesian inference.

The Journal of Open Source Software

State of the field

Dynamical system analysis and modeling is a fundamental tool in many scientific disciplines, and state space models (SSMs) are a widely used framework for modeling and inference in dynamical systems.

Python libraries exist for state space modeling (Aicher et al., 2025; Corenflos & Särkkä, 2021; Johnson, 2020; Lee et al., 2023; S. Linderman et al., 2020; Weiss et al., 2024), which are primarily focused on Hidden Markov Models and discrete-time state space models, with their corresponding Bayesian inference algorithms. Amongst the JAX-native libraries, `dynamax` (S. W. Linderman et al., 2025), `cuthbert` (Corenflos & Duffield, 2026), and `pfjax` (Lysy, 2023) provide low-level frameworks for discrete-time state space modeling and inference. The `rodeo` (Wu & Lysy, 2025) library provides JAX-based probabilistic numerics tools for approximating likelihoods of noisy partially observed data under deterministic continuous-time systems (i.e., ordinary differential equations) but, crucially, does not address randomness in the state evolution.

To the best of our knowledge, there is no existing Python-based low-level library for state space modeling and inference that provides a comprehensive and efficient framework for continuous-discrete state space modeling and inference.

In addition, `dynestyx` (Basis Research Institute, 2025) is a high-level library that stitches many of these low-level libraries (including `cd-dynamax`) together for end-to-end Bayesian inference pipelines for dynamical systems by connecting them with NumPyro (Phan et al., 2019).

Software design

The architecture and design of `cd-dynamax` is driven by our vision of modularity, synergies with existing codebases and extensibility for future research and applications.

We built `cd-dynamax` as a complement and extension to the `dynamax` (S. W. Linderman et al., 2025) library, integrating `diffrax` (Kidger, 2021) to enable continuous-time (CD) dynamics. By leveraging JAX-native numerical differential equation solvers, we achieve seamless hardware acceleration (GPU/TPU) and end-to-end automatic differentiation, which are critical for gradient-based system identification and modern inference algorithms (e.g., Hamiltonian Monte Carlo) that we have incorporated into `cd-dynamax`. This design choice allows us to build on the existing discrete-time SSM modeling and inference tools provided by `dynamax`, while extending its capabilities to handle continuous-time dynamics, irregularly sampled data and nonlinear and non-Gaussian models.

A core design trade-off was balancing compatibility with the discrete-time `dynamax` codebase and the need for flexibility in nonlinear CD-SSM model specification and inference. We settled on a decoupled architecture where CD-SSM model definitions are strictly separated from inference implementations: `cd-dynamax`'s API ensures that practitioners can define a linear/nonlinear CD-SSM once and apply varied inference routines. Additionally, the modular design allows (a) for the integration of new model classes and inference algorithms; and crucially, (b) for the extension of the codebase to interact and be amenable to probabilistic programming interfaces (e.g., NumPyro).

`cd-dynamax`'s internal structure is designed to interact with any CD-SSM model (linear or nonlinear) in a unified way (rather than being treated separately) for model definition, state-inference and system-identification, producing a consistently structured library. This unified structure transforms `cd-dynamax` from a collection of models and scripts into a scalable and extensible framework for CD-SSMs.

We prioritized algorithmic reliability by incorporating a testing suite that validates mathematical correctness against known benchmarks, including CD-SSMs with linear dynamics and discretized

versions of CD-SSMs with nonlinear dynamics. This ensures that the library serves both methodological researchers, who require a stable framework for extending inference theory, and practitioners in fields like systems biology, engineering or finance, who need robust handling of irregularly sampled data.

CD-dynamax modeling and inference framework

Currently, `cd-dynamax` offers:

1. A set of modular definitions of CD-SSM models, capturing both linear (CD-LGSSM) and nonlinear (CD-NLSSM/CD-NLGSSM) dynamics and observation functions with and without assuming Gaussianity, seamlessly incorporating non-regular, noisy observation time instants.
2. Low-level, probabilistic inference algorithms for filtering and smoothing. There exist many algorithms for state inference and parameter estimation in CD-SSMs (Särkkä & Solin, 2019), e.g., Extended Kalman Filter/Smoother, Unscented Kalman Filter/Smoother, Particle Filter/Smoother. Specifically, `cd-dynamax` provides JAX implementations for:
    - Kalman filtering and smoothing for linear Gaussian CD-SSMs.
    - Extended Kalman filtering and smoothing for nonlinear CD-SSMs.
    - Unscented and Ensemble Kalman filtering for nonlinear CD-SSMs.
    - Differentiable Particle Filtering for nonlinear CD-SSMs.
3. A high-level interface for constructing and fitting probabilistic SSMs. We provide readily usable functions for:
    - point-estimation of model parameters via gradient-based or black-box optimization (as in, e.g., `Scipy` (Virtanen et al., 2020), or `Scipy-jaxopt` (Blondel et al., 2021)) of the (approximate) marginal log-likelihood.
    - Bayesian posterior parameter estimation, e.g., Markov Chain Monte Carlo via the BlackJAX (Cabezas et al., 2024) library.

The publicly available `cd-dynamax` documentation and demos provide informative resources describing the use of `cd-dynamax` for CD-SSM modeling, filtering, smoothing, forecasting and fitting to data, for experts and newcomers alike.

Research impact statement

`cd-dynamax`'s research impact spans two complementary scholarly areas: (i) it provides reproducible, illustrative implementations that achieve state-of-the-art results on challenging CD-SSM inference problems, and (ii) it supports active and emerging research collaborations across multiple groups on methodological and applied research in continuous-time dynamical systems modeling and inference.

Benchmarks and reproducible notebooks

The `cd-dynamax` repository includes a suite of tutorial notebooks and demo scripts that achieve state-of-the-art results on known challenging CD-SSM problems. These include:

- **Bayesian parameter uncertainty quantification**: fully Bayesian inference of CD-SSM parameters for the Lorenz-63 system, providing posterior distributions (via MCMC) over physical parameters from noisy observations.
- **Learning chaotic neural SDEs from partial noisy observations**: learning unknown nonlinear dynamics from noisy, partially observed data using neural network parameterizations of the SDE drift, demonstrated on the Lorenz-63 system. We showcase accurate short-term forecasts, as well as accurate long-term statistical forecasts.

- **Sparse identification of dynamical equations**: leverages filtering-based likelihoods to perform Bayesian inference of library-coefficients from sparse, noisy trajectory data, demonstrating competitiveness with SINDy and related approaches (Brunton et al., 2016).

These notebooks serve as reproducible reference implementations and benchmarks for the community, demonstrating `cd-dynamax`'s ability to handle chaotic dynamics, partial observability, and noisy data.

Collaborations and community development

`cd-dynamax` is supporting ongoing research collaborations and open-source developments across multiple groups and use-cases, including:

- Applying CD-SSM inference methodology for patient- and population-level phenotyping via physiologic models. This involves ongoing collaborations with researchers at University of Colorado (studying type-2 diabetes progression) and Dalhousie University and Northwestern University (studying immunologic impacts of pediatric cardiopulmonary bypass surgery). Other similar studies are in earlier planning stages, where patients and populations are thought to be governed by continuous-time dynamical systems with unknown physiologic parameters.
- Applying CD-SSM inference methodology for studying drivers of collective animal behavior from video and acoustic tracking data.
- Supporting graduate research at the Basque Center for Applied Mathematics (BCAM), specifically facilitating Master's theses focused on Bayesian modeling and system identification via the `cd-dynamax` framework.
- `cd-dynamax` is a crucial back-end support for `dynestyx` (Basis Research Institute, 2025), a new high-level library for end-to-end Bayesian inference for dynamical systems based on NumPyro (Phan et al., 2019). This integration validates `cd-dynamax`'s design as a modular, composable low-level library suitable for incorporation into larger probabilistic modeling ecosystems.
- Serving as an implementation substrate in developing new methods for dynamical systems inference (Waxman, 2025).

Several additional collaborations are planned that will be supported by `cd-dynamax`, and we anticipate that these will grow its user base and collaborative community development.

On the importance of continuous-time modeling

While continuous-time SSMs can be represented as discrete-time SSMs when sampling at fixed intervals, there remain fundamental differences between these two modeling paradigms: the former cannot be perfectly translated into the latter without loss of information or introduction of artifacts.

Succinctly put, the relationship between the discrete and continuous frameworks is one of approximation — a mapping that may involve significant information loss: while it is possible to derive a discrete-time model from a continuous-time model through discretization, the reverse process of obtaining a continuous-time model from a discrete-time model is generally ill-posed and non-unique.

There are two fundamental issues introduced by discretization:

- **Information Loss**: Sampling inevitably obscures the system's true dynamics, distorting the signal in a process known as aliasing. Discretization results in the loss of inter-sample behavior, and hence, a system can appear stable at the sampling points while actually experiencing oscillations between them.
- **Artifact Creation**: The choice of a discrete-time representation of a model, along with the definition of its sampling interval, can create non-physical, artificial dynamics.

Discretization choices can introduce entirely new behaviors not present in the original continuous-time system. For instance, naive sampling can induce the emergence (or destruction) of chaos in simple discrete maps (entirely absent, or assured, in their stable continuous-time counterparts) or instability of control-systems (where a stable continuous-time system can be rendered catastrophically unstable by choosing incorrect sampling intervals).

There are **significant benefits of a continuous-time treatment of dynamical systems**:

- *Data agnosticism*: continuous-time models are inherently suited to handle real-world, irregularly-spaced, and missing data: they model the underlying process, not the measurement grid. Thus, continuous-time models naturally generalize to arbitrary observation time grids without retraining or modification.
- *Discretize at the end, not at the beginning*: a continuous-time framing allows for discretization choices to be deferred until the final stages of analysis, enabling the use of adaptive solvers and multi-rate sampling strategies that can better capture the system's dynamics. A history of successes in numerical analysis has shown that delaying discretization until the final stages of computation often leads to more accurate and stable results.
- *Physical interpretability*: continuous-time model parameters represent fundamental, invariant physical properties of the system (e.g., reaction rates, physical constants, clearance rates), whereas discrete-time parameters are a conflation of physical properties and the choices of sampling intervals. In physics-aware modeling, prior knowledge is often most naturally expressed in a continuous-time formulation.
- *First-principles-based theory*: continuous-time models, expressed as differential equations, are the "first principles" foundation for many physical and life sciences. The discrete-time model is most accurately viewed as a subsequent numerical implementation or approximation of this theoretical truth.

AI usage disclosure

The core architectural design, feature development and the algorithmic implementations are the original work of the authors. Core design, scientific and technical decisions and judgments were carried out by the authors.

We acknowledge the use of generative AI to assist in the development and documentation of `cd-dynamax`. Specifically:

- ChatGPT (OpenAI) was used to assist in cleaning code for better readability, writing plotting scripts for visualization, and generating drafts of docstrings and code comments.
- ChatGPT (OpenAI) and Gemini (Google) were employed as general-purpose writing assistants to help polish the narrative, and improve the overall flow and clarity of the paper.

All the codebase and manuscript text were manually reviewed, tested, and edited by the authors. We have verified the correctness of all AI-assisted code, and confirm that the final manuscript accurately reflects our research and design thinking.

Acknowledgements

Iñigo Urteaga acknowledges the support of ''la Caixa'' foundation's LCF/BQ/PI22/11910028 award and Grant RYC2023-045922-I by MICIU/AEI/10.13039/501100011033 and by ESF+, as well as funds by MICIU/AEI/10.13039/501100011033 and the BERC 2022-2025 program

funded by the Basque Government. Matthew Levine acknowledges support by Basis Research Institute and the Eric and Wendy Schmidt Center at the Broad Institute of MIT and Harvard.

References

Aicher, C., Putcha, S., Nemeth, C., Fearnhead, P., & Fox, E. (2025). Stochastic gradient MCMC for nonlinear state space models. *Bayesian Analysis*, *20*(1). https://doi.org/10.1214/23-BA1395

Basis Research Institute. (2025). *Dynestyx: Bayesian inference for dynamical systems*. https://github.com/BasisResearch/dynestyx

Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., & Vert, J.-P. (2021). Efficient and modular implicit differentiation. *arXiv Preprint arXiv:2105.15183*.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). http://github.com/google/jax

Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, *113*(15), 3932–3937. https://doi.org/10.1073/pnas.1517384113

Cabezas, A., Corenflos, A., Lao, J., & Louf, R. (2024). *BlackJAX: Composable Bayesian inference in JAX*. https://arxiv.org/abs/2402.10797

Corenflos, A., & Duffield, S. (2026). *Cuthbert: State-space model inference with JAX*. https://github.com/state-space-models/cuthbert

Corenflos, A., & Särkkä, S. (2021). *Code companion for Bayesian Filtering and Smoothing* (Version 1.0). https://github.com/EEA-sensors/Bayesian-Filtering-and-Smoothing

Johnson, M. J. (2020). *PyHSMM: Bayesian inference in HSMMs and HMMs* (Version 0.0.0). https://github.com/mattjj/pyhsmm

Kidger, P. (2021). *On Neural Differential Equations* [PhD thesis, University of Oxford]. https://docs.kidger.site/diffrax/

Lee, M. A., Yi, B., Martín-Martín, R., Savarese, S., & Bohg, J. (2023). *TorchFilter: Differentiable particle filtering in PyTorch*. https://github.com/stanford-iprl-lab/torchfilter

Linderman, S. W., Chang, P., Harper-Donnelly, G., Kara, A., Li, X., Duran-Martin, G., & Murphy, K. (2025). Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, *10*(108), 7069. https://doi.org/10.21105/joss.07069

Linderman, S., Antin, B., Zoltowski, D., & Glaser, J. (2020). *SSM: Bayesian Learning and Inference for State Space Models* (Version 0.0.1). https://github.com/lindermanlab/ssm

Lysy, M. (2023). *PFJax: Particle filtering in JAX*. https://github.com/mlysy/pfjax

Murphy, K. P. (2023). *Probabilistic machine learning: Advanced topics*. MIT Press. http://probml.github.io/book2

Phan, D., Pradhan, N., & Jankowiak, M. (2019). Composable effects for flexible and accelerated probabilistic programming in NumPyro. *arXiv Preprint arXiv:1912.11554*.

Särkkä, S., & Solin, A. (2019). *Applied stochastic differential equations* (Vol. 10). Cambridge University Press.

Särkkä, S., & Svensson, L. (2023). *Bayesian filtering and smoothing* (Vol. 17). Cambridge

University Press. https://doi.org/10.1017/CBO9781139344203

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waxman, D. (2025). *Sequential Bayesian learning and ensembles for online inference and experimental design* [PhD thesis]. State University of New York at Stony Brook.

Weiss, R., Du, S., Grobler, J., Cournapeau, D., Pedregosa, F., Varoquaux, G., Mueller, A., Thirion, B., Nouri, D., Louppe, G., Vanderplas, J., Benediktsson, J., Buitinck, L., Korobov, M., McGibbon, R., Lattarini, S., Niculae, V., Gramfort, A., Lebedev, S., … Rockhill, A. (2024). *Hmmlearn* (Version 0.3.2). https://github.com/hmmlearn/hmmlearn

Wu, M., & Lysy, M. (2025). *Rodeo: Probabilistic methods of parameter inference for ordinary differential equations*. https://arxiv.org/abs/2506.21776